MonthlyMuse

Technology Stack — detailed reference

AI-Powered Automated Message Generator for Monthly Posting · companion to the Master Blueprint (Sections 13–14) · November 2026

FastAPI backend | Next.js 14 frontend | 16-table relational DB | hosted LLM + adapter chain | own NumPy ranking engine | Vercel + Render | 166 passing tests

1. Stack at a glance

Layer	Choice (installed version)	Why this choice	Blueprint alternative
Backend framework	**FastAPI ≥0.115** + Uvicorn ≥0.30	Same language as the ranking code; async; automatic OpenAPI docs; Pydantic validation on every route	Flask, Django REST, Node (would split the codebase)
Validation / schemas	**Pydantic ≥2.9** + pydantic-settings ≥2.5	Typed request/response models, env-driven config (`MM_` prefix), `.env` support	marshmallow, class-validator
ORM	**SQLAlchemy ≥2.0.30** (declarative)	Mature, explicit query control; portable across SQLite and Postgres	SQLModel, Prisma (JS)
Database	**SQLite** default → **PostgreSQL** via `MM_DATABASE_URL` (psycopg 3 bundled)	Zero-setup local dev and hosting; one env var switches to managed Postgres; vectors stored as float32 BLOBs and cosine computed in NumPy	PostgreSQL + pgvector
LLM generation	**Adapter chain**: Groq (production, `openai/gpt-oss-120b`) · Gemini · OpenAI · Anthropic · Ollama · template (opt-in fallback)	Model-written copy with per-provider isolation: own circuit breaker, model, retry schedule (1/3/9 s + jitter), tolerant JSON parse, cost metering. No single provider outage can take generation down	One hosted model
Web context	**Exa** (`exa-py ≥2.28`), optional via `EXA_API_KEY`	Real-world occasion evidence for the prompt; absence degrades to no web context instead of failing	none specified
NLP preprocessing	**Own `app/ai/text.py`** (normalise, tokenise, lemmas, keywords, n-gram Jaccard, banned-word/CTA checks)	Zero heavy deps, deterministic, CPU-only; blueprint's spaCy/YAKE kept as an optional extra	spaCy small model + YAKE
Embeddings	**sentence-transformers all-MiniLM-L6-v2** (default, 384-d) · **hashing-384** deterministic alternative via `MM_EMBEDDER`	MiniLM gives real semantics in development; the deployed build uses the hashing embedder for instant startup on a small instance. Same dimensionality, so every formula and golden test is unchanged	sentence-transformers all-MiniLM-L6-v2
Numerical / ML	**NumPy ≥1.26 + SciPy ≥1.11** (pure Python algorithms)	Matrix similarity ($E \cdot H^T$, $E \cdot E^T$), softmax logistic-regression tone classifier, Thompson sampling, chi-square, Wilson intervals, Dirichlet Monte-Carlo, PCA — no GPU needed	NumPy, scikit-learn, SciPy
Fallback copy engine	**64 templates across 5 slot banks** (`data/templates.json`)	Guarantees the app is demonstrable offline; used only when `MM_ALLOW_TEMPLATE_FALLBACK=true`, and always labelled in the UI	Hosted model only
Scheduling	**APScheduler ≥3.10** (in-process + standalone worker)	No extra infrastructure; daily cron scan + interval tick; an authenticated webhook tick covers hosts that suspend schedulers	APScheduler; Celery + Redis when scaling

Email	Console backend (dev) · SMTP / Resend options (`MM_EMAIL_BACKEND`)	Free tiers, simple API; in-app notifications always stored in DB	Resend / SendGrid / SMTP (Mailhog in dev)
Auth	**Argon2id** (argon2-cffi) + **JWT** (PyJWT): 15-min access + 14-day refresh httpOnly cookie; signed ICS calendar token	Stateless, standard, secure when cookie flags are right (`None+Secure` in production)	Auth0 / Clerk / Supabase Auth
Frontend	**Next.js 14.2 App Router + TypeScript 5**, React 18	Industry standard, typed contracts, easy deploy, static + dynamic routes	React + Vite
UI toolkit	**Tailwind CSS 3.4** with Section-12 design tokens + own component library (`components/ui.tsx`)	Fast, polished, accessible; tokens (indigo primary, chart palette, text scale, shadows) implemented as Tailwind theme — shadcn/ui CLI dropped to keep the dependency tree auditable	Tailwind + shadcn/ui (MUI, Chakra)
Data fetching	**TanStack Query 5**	Caching, polling (generation status every 1.2 s), invalidation after feedback, error toasts	SWR
Forms	**React Hook Form 7 + Zod 3** (`@hookform/resolvers`)	Schema-validated 3-step Create form with draft autosave	Formik
Charts	**Recharts 3**	Declarative React charts incl. radar (compare mode), scatter (PCA embedding map), donut/bars/CI bars	Chart.js, Plotly
Testing	**pytest** (166 tests, 12 modules) · tsc + ESLint + `next build` for the frontend	Covers golden maths values, pipeline behaviour, occasion gating, plan timing, regeneration, webhooks, API journey, scheduler and the health endpoint	pytest, Playwright, Vitest/RTL
Hosting	**Vercel** (frontend, auto-deploy on push) · **Render** (Docker API service + scheduler worker)	Free tiers that match the stack; the API and worker share one image and differ only in start command	same
CI	**GitHub Actions**: pytest on Python 3.10 + 3.11, frontend typecheck + build on Node 20	Reproducible quality gate on every push and PR	same
Observability	JSON structured logs with `request_id` , stage-by-stage pipeline trace, `/admin/jobs` audit, `/admin/health` with **deployed commit**, LLM latency + cost per request	Cheap visibility into failures and, crucially, into which build is running	Structured logs, Sentry, admin jobs page

2. Architecture (blueprint Section 13)

FRONTEND — Next.js 14 + TypeScript + Tailwind · Vercel
Dashboard · Create · Review/Compare · Calendar · Plans · History · Analytics · Settings — calls only the public API; no secrets in the browser

HTTPS / JSON — JWT access token + refresh cookie ▼

BACKEND API — FastAPI · Render
[Auth + RBAC] → [Validation] → [Routers (58 handlers / 45 paths)] → [Services] → [uniform error / JSON-logging layer]

▼

GENERATION + RANKING — `app/ai/`
preprocess + keywords → context + history retrieval → Thompson-sampled style arms → prompt → **one hosted LLM call (6 candidates)** → hard constraints + dedup → embeddings → features F, s = F·w → MMR top-3 → Monte-Carlo confidence → explanation payload

▼

DATABASE — SQLite (default) / PostgreSQL
16 tables: users, user_preferences, audience_profiles, topics, occasions, monthly_plans, post_cycles, generation_requests, generated_messages, feedback_events, tone_stats, scheduled_posts, post_metrics, notifications, job_runs, analytics_daily

▲

AUTOMATION — APScheduler worker · Render
daily cycle scan → enqueue generation → notify → T-3/T-1 reminders → catch-up → plan extension · job_runs audit · webhook tick for suspended hosts

Statelessness rule from the blueprint is respected: the API holds no in-memory user state, so it can run behind several instances; the worker shares the same code and database but starts with a different command (`python -m app.scheduler.runner`). The only in-memory state that matters — circuit breakers and the scheduler tick lock — is deliberately per-process.

3. Backend in detail

3.1 Runtime & structure

```
backend/app/
├── api/        auth, users, catalog, plans, cycles, messages, analytics, misc  (58 handlers)
├── core/       config.py (MM_ env), errors.py, security.py, logging.py, rate_limit.py, runtime.py
├── db/         session.py (engine, UTCDateTime, blob<->vector helpers), models.py (16 tables)
├── schemas/    Pydantic request/response models
├── ai/         text, embedder, similarity, features, scoring, mmr, confidence, arms, tone,
│               occasion, llm_adapters, prompt, templates, limits, pipeline, personalization,
│               stats, web_search
├── services/   account, catalog, plan, generation, feedback, history, notification, analytics, demo
└── scheduler/  jobs.py, runner.py (APScheduler: daily cron + interval)
```

- **Python 3.10+**, installed with `pip install -e ".[dev]"` from `backend/pyproject.toml`.
- **App lifespan**: create tables → seed catalog (51 occasions, 20 topics, 64 templates) → load/train the tone classifier → start the in-process scheduler (unless disabled) → serve.
- **Async where it matters**: generation runs in a background task (HTTP `202` + polling endpoint) so requests never block; LLM adapters use `httpx.AsyncClient`.
- **Size**: ~7.9k lines of Python across `backend/app`.

3.2 Configuration (pydantic-settings, prefix `MM_`)

Variable	Default	Purpose
`MM_DATABASE_URL`	`sqlite:///./monthlymuse.db`	Swap to `postgresql+psycopg://…`; schema unchanged
`MM_LLM_PROVIDER`	`gemini` (production: `groq`)	`template | gemini | openai | groq | anthropic | ollama`; a bare model name is normalised to its provider
`MM_LLM_MODEL`	provider default	Model for the primary provider; each other provider keeps its own model setting so a Groq name is never sent to Gemini
Per-provider keys	empty	`GROQ_API_KEY`, `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`, Ollama host, `EXA_API_KEY`
`MM_ALLOW_TEMPLATE_FALLBACK`	`false`	**Off by default.** When off, a failed provider marks the request FAILED instead of silently substituting template text
`MM_EMBEDDER`	`sentence-transformers`	`sentence-transformers` (MiniLM, 384-d) or `hash` (deterministic 384-d; used in the deployed build)
`MM_SCHEDULER_ENABLED` / `MM_SCHEDULER_TIMEZONE`	`true` / `Asia/Kolkata`	Worker behaviour and reminder timezone
`MM_SCHEDULER_WEBHOOK_TOKEN`	derived from the secret key	Shared secret for the external-cron tick endpoint
`MM_SECRET_KEY`	dev value	JWT signing — changed per environment, never committed
`MM_GENERATIONS_PER_HOUR` / `MM_REQUESTS_PER_MINUTE`	10 / 100	Sliding-window rate limits
`MM_LLM_MONTHLY_BUDGET_USD`	5.0	Cost guardrail fed by the pricing table
`MM_EMAIL_BACKEND`	`console`	`console | smtp | resend`
`RENDER_GIT_COMMIT`	injected by Render	Read by `/admin/health` so the deployed build is identifiable

3.3 Security

- **Argon2id** password hashing (argon2-cffi), tuned parameters, constant-time compare.
- **JWT access (15 min)** in memory + **refresh (14 days)** in an httpOnly cookie (`SameSite=None; Secure` in production); silent refresh in the API client hides 401s.
- Signed **ICS calendar token** so subscription URLs work without a login; constant-time-compared webhook token for scheduler/seed endpoints.

- Uniform error envelope `{error: {code, message, details, request_id}}`, CORS allow-list, per-user + global rate limits, structured JSON logs carrying `request_id`.

4. AI engine in detail

Generation and ranking are deliberately separate concerns. A **hosted language model** writes the copy; our own **NumPy/SciPy engine** decides which three candidates you see and explains why. That split is what makes the ranking measurable, testable against golden values, and independent of whichever provider is healthy today.

4.1 Provider chain

```
request
  └─ primary provider (MM_LLM_PROVIDER)      e.g. groq → openai/gpt-oss-120b
       ├─ 401/404/429 → Groq model fallback list (gpt-oss-20b, qwen3.8-27b)
       ├─ 429 quota / 5xx → LLMUnavailable → next configured hosted provider
       └─ breaker open (3 fails / 60 s) → skip provider without a call
  └─ every hosted provider failed
       ├─ MM_ALLOW_TEMPLATE_FALLBACK=true  → template engine, batch flagged is_fallback
       └─ otherwise                        → request FAILED, reason stored, previous batch kept
```

Concern	Implementation
One call, six candidates	Over-generate six in a single prompt so candidates are comparable and cost stays at one billing unit per batch
Prompt	System + user prompt carrying brief, style arms (tone × structure), three retrieved exemplars, six recent posts, voice notes, banned words, additional instructions
Provenance	`provider`, `model`, `source`, `created_at`, `fallback_reason` persisted per request and rendered in the review header
Failure semantics	`GenerationUnavailable` is a hard failure by design: nothing is saved, the old batch survives, the UI reports the failure
Cost	Token metering per request, pricing table, monthly budget guardrail

4.2 The fourteen pipeline stages

#	Stage	Module	What happens
1	Validate	`pipeline.py`	Normalise the brief, collect warnings, refuse malformed input before spending a model call
2	Context	`occasion.py`, `text.py`	Occasion admitted only through relevance + date eligibility gates; keywords extracted from topic, occasion and fact
3	Preprocess	`text.py`	Normalise, tokenise, lemma, derive coverage keywords for the hard constraints
4	Intent vector	`embedder.py`	384-d embedding of the brief text; becomes the reference for relevance and the embedding map
5	Retrieval (RAG)	`similarity.py`	History matrix H; top-3 exemplars by cosine + 6 most recent posts
6	Style arms	`arms.py`	Thompson sampling over tone × structure arms; Beta posteriors; 2 exploit + 1 explore
7	Prompt	`prompt.py`	Build system + user prompt from brief, arms, exemplars, recent posts, voice notes
8	Generate	`llm_adapters.py`	One provider call → six candidates; breaker, retry, tolerant parse, cost metering
9	Filter	`limits.py`, `text.py`	Hard constraints: length per platform, banned words, emoji level, CTA, keyword coverage, near-duplicate of history
10	Embed	`embedder.py`, `similarity.py`	L2-normalised candidate embeddings; $E \cdot E^T$ for dedupe and MMR
11	Features	`features.py`	F = [relevance, tone, personalisation, novelty, length, history] $\in [0,1]^6$
12	Score	`scoring.py`	s = F·w against the user's current weight vector (default .30/.20/.20/.15/.10/.05)
13	Rank (MMR)	`mmr.py`	λ = 0.7 with style-arm quotas; the trace records the MMR order and the top-3-by-score it replaced
14	Confidence + explain	`confidence.py`, `stats.py`	200-draw Dirichlet Monte-Carlo → P(rank 1) and band; plain-English reasons; PCA coordinates

4.3 Algorithms and their parameters

Algorithm	Definition	Parameters / notes
Feature extraction	relevance = cos(e, q); tone = softmax classifier P(tone); personalisation = ⟨e, u); novelty = 1 − max cos(e, H); length = clipped z-score against learned stats; history = Beta posterior mean	Every feature mapped into [0,1] so the score is interpretable and weights are comparable
Scoring	$s = F \cdot w$	default w = (.30, .20, .20, .15, .10, .05), always non-negative and summing to 1
Constrained gradient step	$w \leftarrow \text{proj}_{\text{simplex}}(w + \eta \cdot \nabla_w)$, $\nabla_w = -\beta(f_{\text{chosen}} - \Sigma_k\, p_k f_k) + 2\lambda(w - w_0)$	Softmax choice model with inverse temperature β = 8, regularisation λ = 0.005 pulling back toward the default weights, learning rate η = 0.01; projection keeps w non-negative and summing to 1 after every update
Preference vector	$u \leftarrow (1 - \alpha)\cdot u + \alpha \cdot e_{\text{chosen}}$	Exponential moving average over chosen/copied candidates; drives the personalisation feature
Thompson sampling	$\theta_{\text{arm}} \sim \text{Beta}(\alpha, \beta)$; pick the top three draws	Laplace-smoothed priors per (tone, structure) arm; 2 exploit + 1 explore per batch
MMR	$\text{argmax}_d\, [\lambda \cdot \text{rel}(d) - (1 - \lambda)\cdot \max_{s \in S} \text{sim}(d, s)]$	λ = 0.7; greedy and exhaustive variants; style-arm quota prevents three variants of one style
Confidence	P(rank 1) from 200 Dirichlet draws around w, scoring the feature matrix each draw	Seed 42; high / medium / low bands; deterministic for fixed inputs
Wilson interval	$(\hat{p} + z^2/2n \pm z\sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)}) / (1 + z^2/n)$	95% default; selection rate, approval rate, top-1 agreement
Tone classifier	softmax(Wx + b) over eight tone labels	800 labelled rows, trained at startup, weights cached to `tone_training_model.npz`
Deduplication	n-gram Jaccard similarity above threshold → drop	Opening-line and full-text variants; near-duplicate check against published history
Occasion gating	relevance = |name tokens ∩ objective tokens| / |name tokens| ≥ 0.5, or a festive-intent bridge score of 0.5; plus date eligibility	Pure and clock-explicit — the caller supplies today in the user's timezone; a past occasion never enters context automatically
Analytics statistics	χ^2 independence, Shannon entropy, MRR, top-1 agreement, Pearson correlation	All computed per user over their own history, with small-n caveats shown in the UI
Dimensionality reduction	PCA to 2-D for the embedding map	Displayed candidates + intent point + history points, so the user can see the candidate cloud

> **Determinism:** seeded RNG everywhere (Monte-Carlo, Thompson, tone training) — same inputs and weights always produce the same ranking, which is what makes the golden-value suite possible. Model text is not deterministic, which is exactly why the *ranking* and its explanation are the parts under test.

5. Frontend in detail

- **Next.js 14.2 (App Router) + TypeScript 5** — 12 routes (10 product screens plus the root redirect and the review resolver); ~3.9k lines; client components only where interactivity demands it.
- **Tailwind CSS 3.4** implements the Section 12 design system: indigo primary scale, 7-colour chart palette, type scale, radii, shadows, plus component classes (`card` , `btn` , `input` , `chip` , `badge`) and reduced-motion support.
- **TanStack Query 5** — one `http` wrapper (`lib/api.ts`) handles JSON, auth headers, silent refresh on 401 and typed errors; generation status polls every 1.2 s until done.
- **React Hook Form + Zod** — the 3-step Create form validates with a Zod schema shared by all steps and autosaves drafts.
- **Inline SVG** for score rings and badges; **Recharts 3** for the radar (compare), scatter (embedding map), donut/bars/line/CI bars (analytics); **lucide-react** icons; **date-fns** formatting.
- **Review screen** shows provenance (provider, model, source, timestamp), the saved brief, and a labelled banner when a batch came from the fallback engine. The candidate cards carry the score ring, confidence chip, similarity meter, why-ranked reasons, expandable feature bars, compare and embedding-map views, and select / edit / copy / like / dislike / reject / regenerate.

Screen	Blueprint section	What it does
Dashboard	3.1	Next-action banner, this-month card, 12-cycle timeline, quick stats, activity feed
Create	3.2	3-step form, tone chips with Bayesian recommendation hints, occasion/keyword chips, "what the AI will use" panel
Review / Compare	3.3	Provenance header, saved brief, 3 ranked cards with score rings, confidence, similarity, why-ranked, feature bars, radar compare, PCA map, filtered-candidate disclosure, regenerate-all, candidate actions
Calendar	3.4	Month grid with status chips, cycle modal (generate now / skip / reschedule), .ics download
Plans	3.7	List/create/pause/resume/delete Monthly Plans
History	3.5	Text + semantic search, filters, detail drawer with batch context, reuse
Analytics	3.6	Usage / Content / Quality tabs: donut, bars, lines, Wilson CI bars, chi-square panel, novelty & entropy
Settings	3.7	Voice, banned words, emoji, length, variety slider, pause learning, learned priorities + tone arms + reset, audiences CRUD, export/delete

6. Data layer

- **16 tables** per blueprint Section 9 (listed in §2); portable string enums instead of native PG enums so SQLite and Postgres share one schema.
- **UTC everywhere**: a custom `UTCDateTime` TypeDecorator normalises SQLite's naive datetimes back to aware UTC — no naive/aware mixing bugs.

- **Vectors** as float32 BLOBs (`vec_to_blob/blob_to_vec`); cosine computed in NumPy — identical results on SQLite and Postgres, pgvector-ready for scale.
- **JSON columns** where the blueprint allows them: per-request stage trace, six-feature score breakdowns, tone counts, the saved brief and provenance.
- **Portability**: `psycopg[binary]` is a first-class dependency and plain `postgresql://` URLs are normalised to `postgresql+psycopg://` with TLS, so managed Postgres is a configuration change.

7. Automation (worker)

- **APScheduler ≥3.10**: daily cron (timezone `MM_SCHEDULER_TIMEZONE`) + interval trigger; runs in-process by default, or standalone via `python -m app.scheduler.runner` (compose and Render do the latter).
- Jobs: `prepare_due_cycles` (idempotent — creates the next cycle + generation request), `send_reminders` (T-3, T-1, NEEDS_ATTENTION nudges), `catch_up` (missed runs), `extend_plans` (auto-extend the 12-cycle horizon), all recorded in `job_runs` for the admin audit.
- Hosts that suspend background schedulers can drive the same idempotent tick through `POST /api/v1/admin/scheduler/run` with a shared secret; concurrent ticks are rejected with 409.
- Cycle state machine: `PLANNED → GENERATING → AWAITING_REVIEW → APPROVED → SCHEDULED → PUBLISHED` (+ SKIPPED / FAILED).

8. Testing, CI, DevOps

Suite	Tests	Contents
`test_math.py`	35	Section 17 golden values: feature/score vector, gradient steps (w_N→0.161, w_P→0.195, w_L→0.094), MMR order [A,C,F], Wilson CI (3,10)=(0.108,0.603), choice probabilities ≈[0.46,0.34,0.20]
`test_pipeline_behavior.py`	29	Provider-aware generation, no silent template substitution, explicit fallback opt-in, provenance fields, filtered candidates, seeded confidence
`test_occasion_gates.py`	22	Relevance threshold, festive bridge, unrelated objectives, past-date eligibility, explicit retrospective
`test_plan_timing.py`	19	Cycle dates, timezone handling, explicit start dates independent of the real clock, horizon filling
`test_pipeline.py`	13	Hard constraints, dedup, arm-aware MMR, stage payload shape
`test_text_quality.py`	12	Copy quality: placeholder leakage, banned words, CTA, length bounds
`test_api.py`	11	End-to-end journey: register → plan → generate (202 + poll) → feedback → approve → analytics
`test_monthly_regeneration.py`	6	Regeneration uses the configured provider and the saved brief; failure keeps the previous batch
`test_webhook.py`	6	Webhook auth, idempotency, concurrent-tick rejection
`test_demo_accounts.py`	6	Seeded personas, idempotent seeding, integrity
`test_scheduler.py`	4	Reminder days, idempotent preparation, catch-up, plan extension
`test_health_commit.py`	3	Deployed commit from `RENDER_GIT_COMMIT`: set, unset (null) and fallback handling
Total	**166**	**All passing**

CI (`.github/workflows/ci.yml`): backend pytest on Python 3.10 & 3.11; frontend `tsc --noEmit` + production build on Node 20 — every push and PR. **Compose** (`docker-compose.yml`): postgres + api + worker + frontend, health-checked, one command start. **Deployment**: Vercel (frontend, auto-deploy) and Render (API + worker, Docker), with `/admin/health` reporting the running commit.

9. Deviations from the blueprint (and why)

Blueprint recommendation	What we shipped	Reason
PostgreSQL + pgvector	SQLite default, Postgres-ready	Zero-setup development and hosting; one env var switches, schema & tests identical. Managed Postgres is the next durability step
sentence-transformers (384-d)	MiniLM in development, deterministic hashing embedder in the deployed build	Instant startup on a small instance; same dimensionality so every formula and test is unchanged
spaCy + YAKE preprocessing	Own `text.py` (spaCy as an optional extra)	Heavy native deps; the needed operations are small and testable in isolation
shadcn/ui	Own component library on Tailwind tokens	Fewer moving parts, full control of the Section 12 tokens, no CLI-generated drift
Hosted LLM in the hot path	**Adopted**: Groq in production behind a five-provider adapter chain	Model-written copy is materially better; the template engine survives as an opt-in, clearly labelled fallback rather than the default

10. Runtime requirements

- **Dev machine**: Python 3.10+ (tested 3.10.11), Node 18+ (tested Node 20/22). No Docker or PostgreSQL needed for SQLite mode.
- **Production**: any Docker host — `docker compose up --build`; or Vercel + Render as deployed (set `NEXT_PUBLIC_API_URL`, `MM_ENVIRONMENT=production`, a strong `MM_SECRET_KEY` and the provider key).
- **Scale path**: swap `MM_DATABASE_URL` → managed Postgres, run multiple stateless API instances, move the worker to its own service (already supported), and enable MiniLM for stronger semantics — no code changes required.

MonthlyMuse · Technology Stack reference · generated from the implemented codebase (58 route handlers / 45 paths, 166 tests, 12 frontend routes, 16 tables) · November 2026